Docker & Kubernetes for Developers

Session 07: Kubernetes'te DevOps Pratikleri

Eğitmen: Hüseyin Babal

Süre: 2 Saat (Teori + Hands-on Lab)

📋 Bu Oturum İçeriği

- ✅ CI/CD fundamentals ve Kubernetes
- ✅ GitHub Actions ile automated pipelines
- ✅ GitOps principles ve best practices
- ✅ FluxCD ile continuous deployment
- ✅ Container registry integration
- ✅ **Complete automation for e-commerce (Lab)**
- ✅ Security ve compliance

🔄 Traditional vs Modern Deployment

Traditional Deployment (Push Model)

```
Developer → CI/CD Pipeline → Production
     ↓              ↓              ↓
   Git Push → Build & Test → kubectl apply
```

GitOps (Pull Model)

```
Developer → Git Repository → GitOps Agent → Kubernetes
     ↓              ↓                ↓              ↓
  Git Push → Config Change → Sync & Deploy → Desired State
```

GitOps = Git + Operations

🎯 GitOps Principles

4 Temel Prensip:

1️⃣ Declarative Configuration

- Infrastructure ve applications **YAML** olarak tanımlanır
- **Git repository** single source of truth

2️⃣ Version Controlled

- **Her değişiklik** Git'te tracked
- **Rollback** capability her zaman

3️⃣ Automated Deployment

- **GitOps agent** otomatik sync yapar

🏗️ CI/CD Pipeline Architecture

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Source      │    │   CI         │    │   CD         │
│  Control     │ -> │  Pipeline    │ -> │  Pipeline    │
│  (Git)       │    │  (Build)     │    │  (Deploy)    │
└──────────────┘    └──────────────┘    └──────────────┘
       |                   |                   |
       v                   v                   v
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ Code Change  │    │Docker Image  │    │ Kubernetes   │
│ Detection    │    │Build & Push  │    │ Deployment   │
└──────────────┘    └──────────────┘    └──────────────┘
```

CI (Continuous Integration):

- Code quality checks
- Automated testing

🚀 GitHub Actions Workflow

```yaml
name: E-commerce CI/CD

on:
  push:
    branches: [main]
    paths: ["microservices/**"]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [user-service, product-service, order-service]

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Log in to Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          context: ./microservices/${{ matrix.service }}
          push: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/${{ matrix.service }}:${{ github.sha }}
```

🧪 Lab: GitHub Actions Setup

Step 1: Repository Structure

```
.github/workflows/
├── ci-user-service.yml
├── ci-product-service.yml
├── ci-order-service.yml
└── deploy.yml

k8s-manifests/
├── user-service/
├── product-service/
├── order-service/
└── infrastructure/
```

Step 2: Service-specific Workflow

```yaml
name: User Service CI
```

🔄 FluxCD - GitOps Engine

FluxCD Kurulumu

```
# Flux CLI kurulum
curl -s https://fluxcd.io/install.sh | sudo bash

# Bootstrap Flux
flux bootstrap github \
  --owner=huseyinbabal \
  --repository=k8s-docker-for-developers-training \
  --branch=main \
  --path=./clusters/production \
  --personal
```

GitRepository Resource

```
apiVersion: source.toolkit.fluxcd.io/v1beta2
kind: GitRepository
```

🎯 FluxCD Kustomization

```yaml
apiVersion: kustomize.toolkit.fluxcd.io/v1beta2
kind: Kustomization
metadata:
  name: ecommerce-app
spec:
  interval: 10m
  sourceRef:
    kind: GitRepository
    name: ecommerce-source
  path: "./k8s-manifests"
  prune: true
  timeout: 5m
  targetNamespace: ecommerce
  healthChecks:
    - apiVersion: apps/v1
      kind: Deployment
      name: user-service
    - apiVersion: apps/v1
      kind: Deployment
      name: product-service
    - apiVersion: apps/v1
      kind: Deployment
      name: order-service
```

🏪 Lab: Complete E-commerce Pipeline

Kustomization Structure

```
k8s-manifests/
├── kustomization.yaml
├── namespace.yaml
├── infrastructure/
│   ├── kustomization.yaml
│   ├── mysql.yaml
│   ├── postgresql.yaml
│   ├── mongodb.yaml
│   └── rabbitmq.yaml
└── applications/
    ├── kustomization.yaml
    ├── user-service/
    │   ├── kustomization.yaml
    │   ├── deployment.yaml
    │   ├── service.yaml
    │   └── configmap.yaml
```

🔧 Lab: Kustomize Configuration

Base Kustomization

```yaml
# k8s-manifests/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - namespace.yaml
  - infrastructure/
  - applications/

images:
  - name: user-service
    newName: ghcr.io/huseyinbabal/k8s-docker-for-developers-training/user-service
    newTag: latest
  - name: product-service
    newName: ghcr.io/huseyinbabal/k8s-docker-for-developers-training/product-service
    newTag: latest
  - name: order-service
    newName: ghcr.io/huseyinbabal/k8s-docker-for-developers-training/order-service
    newTag: latest
```

🚀 Lab: Automated Image Updates

ImageUpdateAutomation

```yaml
apiVersion: image.toolkit.fluxcd.io/v1beta1
kind: ImageRepository
metadata:
  name: user-service-image
spec:
  image: ghcr.io/huseyinbabal/k8s-docker-for-developers-training/user-service
  interval: 1m
```

```yaml
apiVersion: image.toolkit.fluxcd.io/v1beta1
kind: ImagePolicy
metadata:
  name: user-service-policy
spec:
  imageRepositoryRef:
    name: user-service-image
  policy:
    semver:
      range: ">=1.0.0"
```

```yaml
apiVersion: image.toolkit.fluxcd.io/v1beta1
kind: ImageUpdateAutomation
metadata:
  name: ecommerce-image-update
spec:
  interval: 30m
  sourceRef:
    kind: GitRepository
    name: ecommerce-source
  git:
    checkout:
      ref:
        branch: main
    commit:
      author:
        name: fluxcdbot
        email: fluxcdbot@users.noreply.github.com
      messageTemplate: "Update images to {{range .Updated.Images}}{{println .}}{{end}}"
    push:
      branch: main
  update:
    path: "./k8s-manifests"
    strategy: Setters
```

🔐 Security Best Practices

Container Security

```yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  fsGroup: 2000
  seccompProfile:
    type: RuntimeDefault
  capabilities:
    drop:
      - ALL
```

Network Policies

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: ecommerce-network-policy
spec:
  podSelector:
    matchLabels:
      app: user-service
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: api-gateway
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: mysql
```

📊 Lab: Monitoring Pipeline

Prometheus ServiceMonitor

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: ecommerce-metrics
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: ecommerce
  endpoints:
    - port: metrics
      interval: 30s
      path: /actuator/prometheus
```

Grafana Dashboard

🎭 Lab: Blue-Green Deployment

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: user-service-rollout
spec:
  replicas: 4
  strategy:
    blueGreen:
      activeService: user-service-active
      previewService: user-service-preview
      autoPromotionEnabled: false
      scaleDownDelaySeconds: 30
      prePromotionAnalysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: user-service-preview
      postPromotionAnalysis:
        templates:
          - templateName: success-rate
        args:
          - name: service-name
            value: user-service-active
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: user-service:v1.0.0
```

🔄 Lab: Canary Deployment

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: product-service-rollout
spec:
  replicas: 10
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 2m }
        - setWeight: 30
        - pause: { duration: 5m }
        - analysis:
            templates:
              - templateName: error-rate
            args:
              - name: service-name
                value: product-service
        - setWeight: 60
        - pause: { duration: 10m }
        - setWeight: 100
      canaryService: product-service-canary
      stableService: product-service-stable
      trafficRouting:
        nginx:
          stableIngress: product-service-stable
          annotationPrefix: nginx.ingress.kubernetes.io
```

📝 Sonraki Oturum Önizlemesi

Session 08: Gözlemlenebilirlik ve Sorun Giderme

- **Logging** with Grafana Loki
- **Metrics** with Prometheus & Grafana
- **Distributed tracing** with Grafana Tempo
- **Istio service mesh**
- **Complete observability stack** for e-commerce

Hazırlık:

- Monitoring concepts gözden geçirin
- Grafana basics öğrenin

🙏 Teşekkürler!

DevOps Master! 🚀

Bugün öğrendiklerimiz:

✅ GitHub Actions ile CI/CD pipelines kurduk
✅ GitOps principles öğrendik
✅ FluxCD ile automated deployment
✅ Security best practices uyguladık
✅ Blue-Green ve Canary deployment patterns

İletişim:

- GitHub: @huseyinbabal

Sonraki Oturum: Session 08 - Gözlemlenebilirlik ve Sorun Giderme